

Experiment 1

Question:- Implementation of Matrix Multiplication and Linear Algebra Operations using NumPy.

Code:

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
result = np.dot(A, B)
print("Matrix Multiplication Result:\n", result)
print("VARUN PAWAR  ")
```

Output:

Matrix Multiplication Result:

```
[[19 22]
 [43 50]]
```

VARUN PAWAR

Experiment 2

Question:- Statistical Data Analysis: Calculating Mean, Median, and Mode using NumPy and SciPy.

Code:

```
import numpy as np
from scipy import stats
data = [10, 20, 20, 30, 40, 50, 60]
print(f"Mean: {np.mean(data)}")
print(f"Median: {np.median(data)}")
print(f"Mode: {stats.mode(data, keepdims=True).mode[0]}")
print("VARUN PAWAR  ")
```

Output:

Mean: 32.857142857142854

Median: 30.0

Mode: 20

VARUN PAWAR

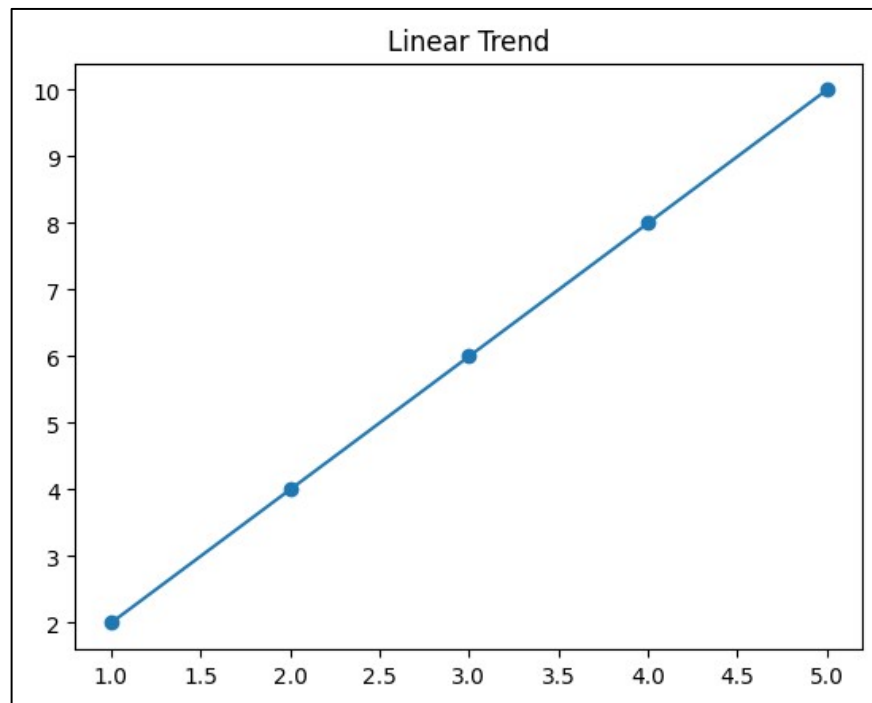
Experiment 3

Question:- Introduction to Data Visualization: Plotting Basic Linear Trends.

Code:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y, marker='o')
plt.title("Linear Trend")
plt.show()
print(" VARUN PAWAR  ")
```

Output:



VARUN PAWAR

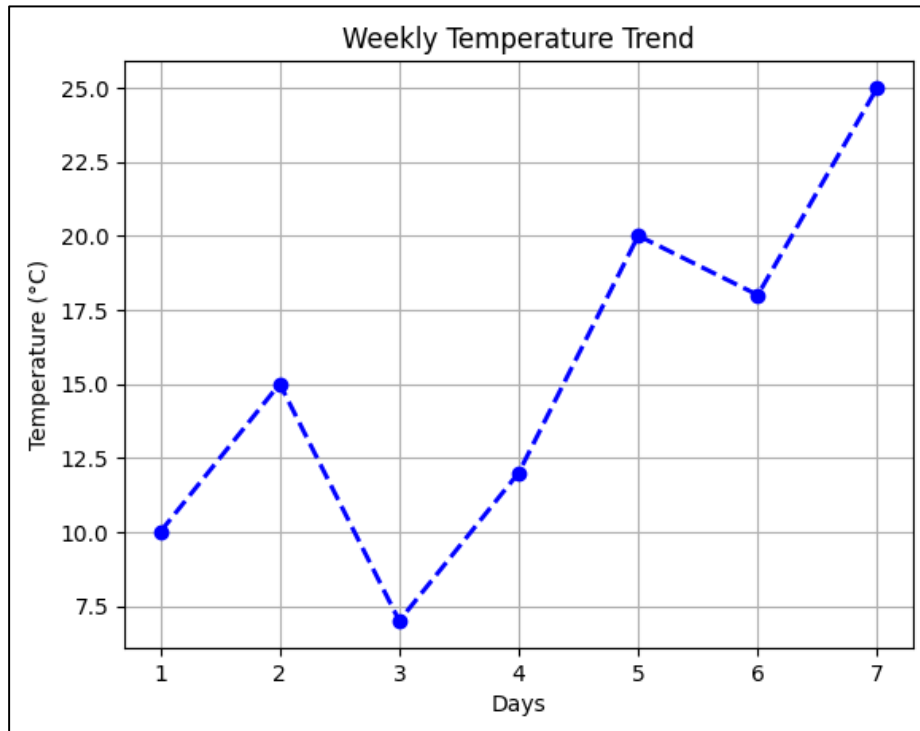
Experiment 4

Question:- Time-Series Visualization: Customizing Line Plots for Weekly Trends.

Code:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5, 6, 7] #Days
y = [10, 15, 7, 12, 20, 18, 25] # Temperature
plt.plot(x, y, color='blue', marker='o', linestyle='dashed', linewidth=2)
plt.title("Weekly Temperature Trend")
plt.xlabel("Days")
plt.ylabel("Temperature (°C)")
plt.grid(True)
plt.show()
print("VARUN PAWAR  ")
```

Output:



VARUN PAWAR

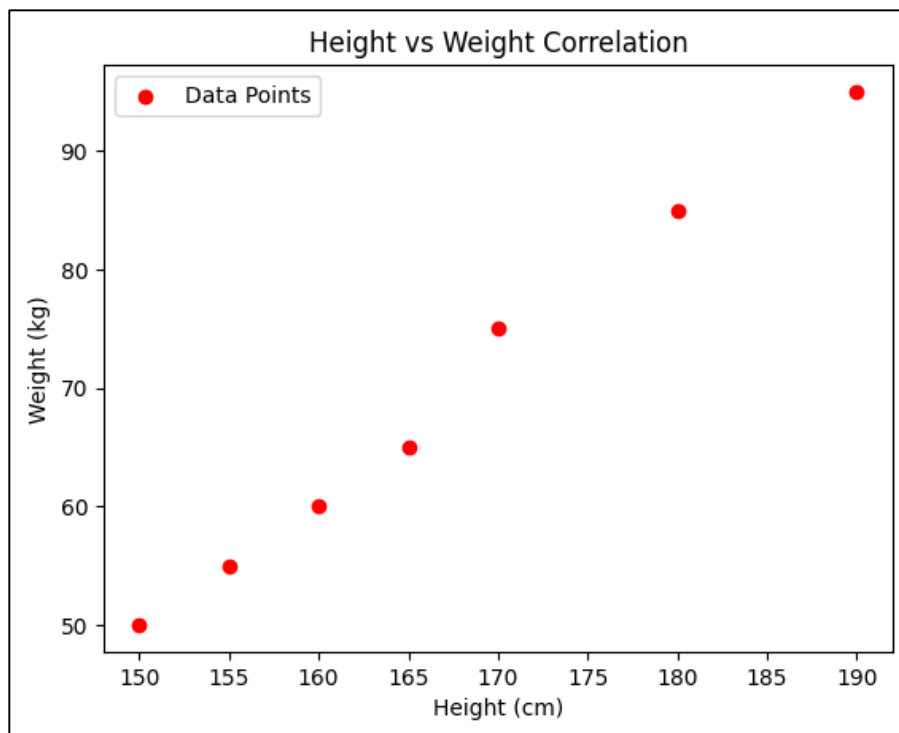
Experiment 5

Question:- Correlation Analysis: Creating Scatter Plots to Visualize Relationships between Features.

Code:

```
import matplotlib.pyplot as plt
height = [150, 160, 170, 180, 190, 155, 165]
weight = [50, 60, 75, 85, 95, 55, 65]
plt.scatter(height, weight, color='red', label='Data Points')
plt.title("Height vs Weight Correlation")
plt.xlabel("Height (cm)")
plt.ylabel("Weight (kg)")
plt.legend()
plt.show()
print("VARUN PAWAR  ")
```

Output:



VARUN PAWAR

Experiment 6

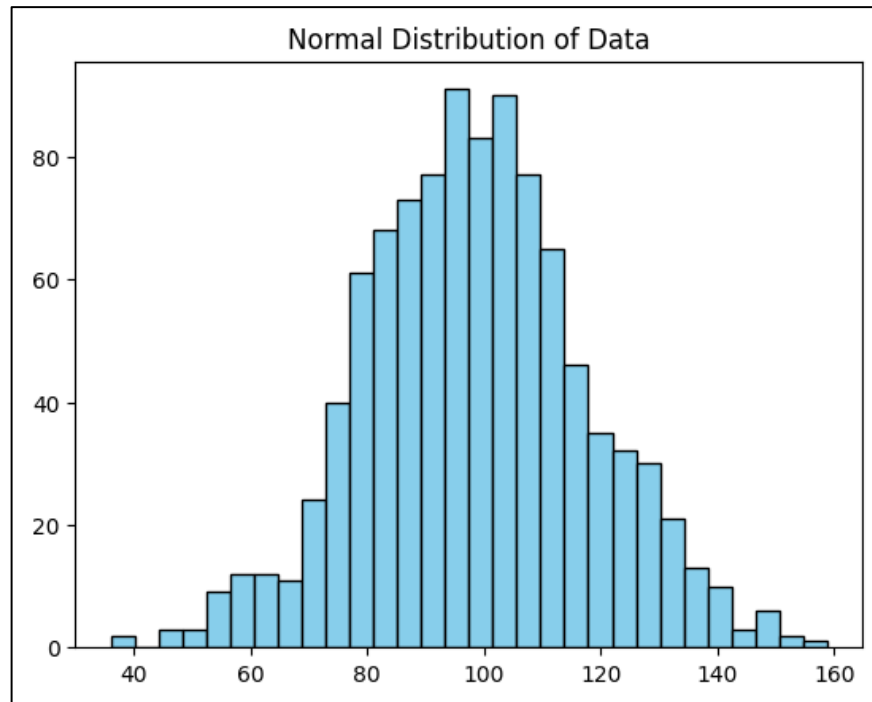
Question:- Distribution Analysis: Generating Histograms and Modeling Normal

Distributions.

Code:

```
import matplotlib.pyplot as plt
import numpy as np
data = np.random.normal(100, 20, 1000) # 1000 random numbers generate karna
plt.hist(data, bins=30, color='skyblue', edgecolor='black')
plt.title("Normal Distribution of Data")
plt.show()
print("VARUN PAWAR ")
```

Output:



VARUN PAWAR

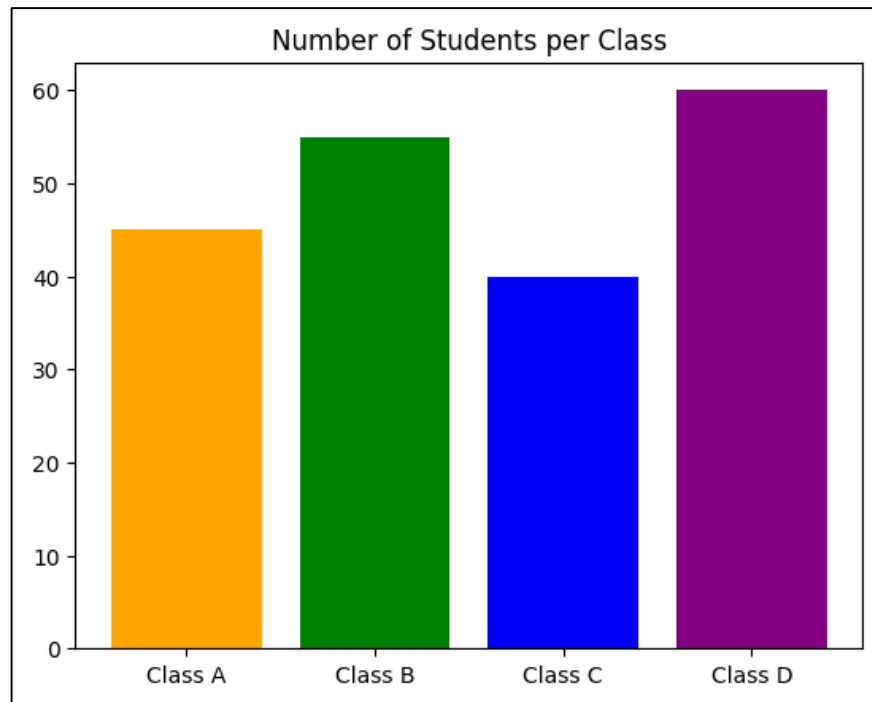
Experiment 7

Question:- Comparative Analysis: Visualizing Categorical Data using Bar Charts.

Code:

```
import matplotlib.pyplot as plt
classes = ['Class A', 'Class B', 'Class C', 'Class D']
students = [45, 55, 40, 60]
plt.bar(classes, students, color=['orange', 'green', 'blue', 'purple'])
plt.title("Number of Students per Class")
plt.show()
print("VARUN PAWAR  ")
```

Output:



VARUN PAWAR

Experiment 8

Question:- Comparative Analysis: Visualizing Categorical Data using Bar Charts.

Code:

```
from sklearn.linear_model import LinearRegression
import numpy as np
X = np.array([[1], [2], [3], [4]]) # Hours
y = np.array([10, 20, 30, 40]) # Score
model = LinearRegression().fit(X, y)
print(f'Prediction for 5 hours: {model.predict([[5]])[0]}')
print("VARUN PAWAR")
```

Output:

Prediction for 5 hours: 50.0

VARUN PAWAR

Experiment 9

Question:- Binary Classification using Logistic Regression for Probability-Based

Outcomes.

Code:

```
from sklearn.linear_model import LogisticRegression
X = [[10], [20], [30], [40], [50]] # Scores
y = [0, 0, 0, 1, 1] # 0=Fail, 1=Pass
clf = LogisticRegression().fit(X, y)
print(f'Predict for score 45: {clf.predict([[45]])}')
print("VARUN PAWAR ")
```

Output:

Predict for score 45: [1]

VARUN PAWAR

Experiment 10

Question:- Data Preprocessing: Implementing Train-Test Split for Model Evaluation.

Code:

```
from sklearn.model_selection import train_test_split
import numpy as np
X, y = np.arange(10).reshape((5, 2)), range(5)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
print(f"Train size: {len(X_train)}, Test size: {len(X_test)}")
print("VARUN PAWAR  ")
```

Output:

Train size: 4, Test size: 1

VARUN PAWAR

Experiment 11

Question:- Classification using Support Vector Machines (SVM) with Linear Kernels.

Code:

```
from sklearn.svm import SVC
X = [[0, 0], [1, 1]]
y = [0, 1]
clf = SVC(kernel='linear').fit(X, y)
print(f"SVM Prediction: {clf.predict([[0.5, 0.6]])}")
print("VARUN PAWAR  ")
```

Output:

SVM Prediction: [1]

SVM Prediction: [1]

VARUN PAWAR

Experiment 12

Question:- Model Evaluation Metrics: Performance Assessment using the Confusion

Matrix.

Code:

```
from sklearn.metrics import confusion_matrix
y_true = [0, 1, 0, 1]
y_pred = [1, 1, 0, 1]
print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
print("VARUN PAWAR  ")
```

Output:

Confusion Matrix:

```
[[1 1]
 [0 2]]
```

VARUN PAWAR

Experiment 13

Question:- Visualizing Neural Network Components: Implementation of the Sigmoid

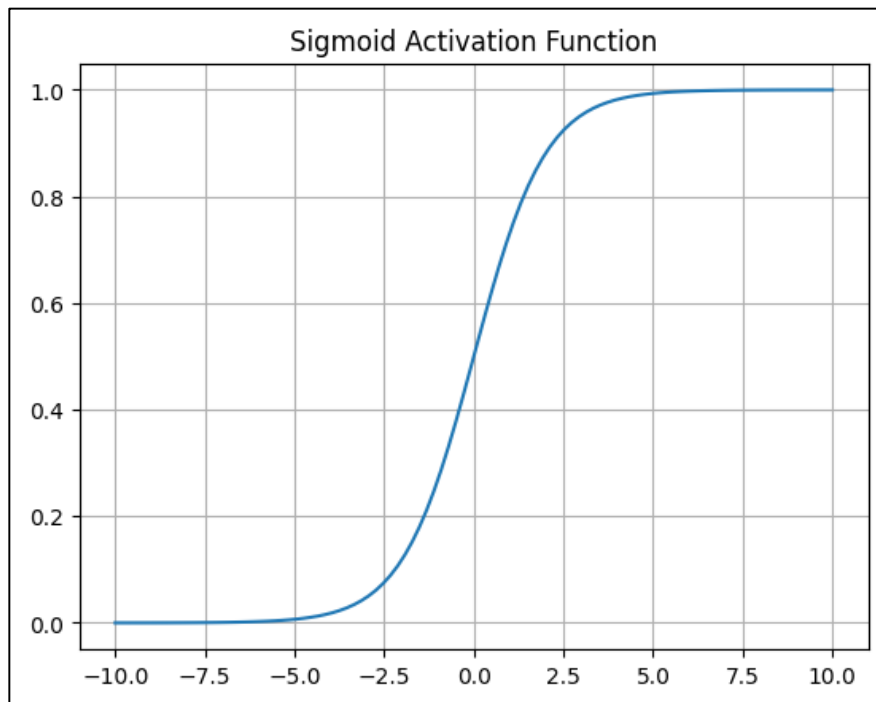
Activation Function.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

x = np.linspace(-10, 10, 100)
plt.plot(x, sigmoid(x))
plt.title("Sigmoid Activation Function")
plt.grid()
plt.show()
print("VARUN PAWAR  ")
```

Output:



VARUN PAWAR

Experiment 14

Question:- Non-Linearity in Deep Learning: Implementing the Rectified Linear Unit (ReLU) Function.

Code:

```
def relu(x):  
    return np.maximum(0, x)  
  
test_data = np.array([-5, -2, 0, 3, 7])  
print("ReLU Input:", test_data)  
print("ReLU Output:", relu(test_data))  
print("VARUN PAWAR  ")
```

Output:

```
ReLU Input: [-5 -2  0  3  7]  
ReLU Output: [0 0 0 3 7]
```

VARUN PAWAR

Experiment 15

Question:- Natural Language Processing (NLP) Preprocessing: Text Tokenization and Word Indexing.

Code:

```
from tensorflow.keras.preprocessing.text import Tokenizer

sentences = ["Machine Learning is great", "I love Deep Learning"]
tokenizer = Tokenizer(num_words=100)
tokenizer.fit_on_texts(sentences)
print("Word Index:", tokenizer.word_index)
print("VARUN PAWAR  ")
```

Output:

Word Index: {'learning': 1, 'machine': 2, 'is': 3, 'great': 4, 'i': 5, 'love': 6, 'deep': 7}

VARUN PAWAR

Experiment 16

Question:- Sentiment Analysis: Determining Text Polarity using NLP Libraries.

Code:

```
from textblob import TextBlob
text = "Machine Learning is absolutely amazing and fun!"
blob = TextBlob(text)
sentiment = blob.sentiment.polarity # -1 (Neg) to 1 (Pos)
result = "Positive" if sentiment > 0 else "Negative"
print(f"Text: {text} | Sentiment: {result}")
print("VARUN PAWAR  ")
```

Output:

Text: Machine Learning is absolutely amazing and fun! | Sentiment: Positive

VARUN PAWAR